What Remains Hard after ETH?

Exact Counting, Better Representations, and a Refined Hardness Question

Rohan Vinaik

7 November 2026

Abstract

A subexponential algorithm can overturn ETH while leaving a very strong form of computational hardness intact. The distinction is between shaving logarithmic factors from an exponent and reducing that exponent by a fixed power. This note explains the public successor hypothesis for exact counting, why the task and its representation matter, and what a Lean-verified finite tradeoff establishes for one algebraic counting construction. It accompanies the technical article, *Refining Hardness after ETH*.

1 A boundary can fail without the whole landscape collapsing

The Exponential Time Hypothesis made a precise prediction about satisfiability: no deterministic algorithm would solve 3-SAT in $2^{o(n)}$ time, apart from polynomial input factors, where n is the number of variables. The recent OpenAI and Claude papers report algorithms that cross this boundary, including algorithms that compute the exact number of satisfying assignments [1, 5]. If those analyses are correct, the original ETH is false.

But the boundary was specific. Consider the difference between

$$2^{n/\sqrt{\log n}} \qquad \text{and} \qquad 2^{\sqrt{n}}.$$

Both are subexponential. Their gains are very different. In the first, the exponent remains larger than $n^{0.99}$, or indeed $n^{1-\varepsilon}$ for any fixed positive ε, once n is sufficiently large. In the second, its power has dropped to one half.

That leaves a strong question after ETH: can exact counting achieve a fixed-power improvement in its exponent, or does its worst-case complexity retain the scale $2^{n^{1-o(1)}}$?

This question already has a public mathematical formulation. The counting Satisfiability Hypothesis, #SH, excludes exact CNF-counting algorithms with running time $O(2^{L^{1-\varepsilon}})$ for every fixed $\varepsilon > 0$, where L counts the formula's variables, clauses, and literal occurrences. MIT Hardness Group proves that it is equivalent to Weak #ETH, the corresponding variable-based hypothesis for #3-SAT with polynomial input factors [4]. The hypothesis remains unproved. The reported logarithmic improvements in the exponent do not, by themselves, contradict it.

The technical article accompanying this note asks what structural reasoning could support such a refined hardness claim [7]. Its aim is to identify the work an exact computation still has to do after a better representation has delivered a substantial gain.

2 What does the answer require us to distinguish?

An exact count depends on which assignments satisfy a formula. It does not follow that every assignment must be checked separately. Algebraic summation, decomposition, and symmetry can combine many contributions in one computation. Any argument for hardness has to account for those joint operations.

A small example makes the role of access clear. Suppose we can inspect a Boolean function only by asking whether a chosen assignment satisfies it. If exactly one assignment satisfies the function, seeing that positive answer certifies satisfiability immediately. It does not certify an exact count of one. Every uninspected assignment might be another satisfying assignment.

This difference persists even when all the alternative functions have short width-two CNF descriptions. A singleton satisfying set needs only unit clauses. A two-point satisfying set can be expressed with fixed coordinates and signed equivalences. Under the declared point observations, exact counting needs all 2^n answers, while satisfiability at a singleton needs one.

The example establishes an information requirement for those observations. It does not establish exponential running time for a program given the actual formula. Reading an explicit clause list provides different information. In fact, reading all the bits of an input acquires the whole input; the remaining difficulty lies in computing with it.

This is the first lesson for a refined account of hardness. The task determines which alternatives matter. The permitted operations determine which alternatives can be resolved together. Their implementation determines the computational cost. Counting candidates, or counting certificates alone, does not settle all three questions.

A change of representation can therefore be a real algorithmic advance even when it preserves every distinction relevant to the answer. It may simply make those distinctions much cheaper to resolve jointly. Transferring an old lower bound requires a stated relation between the old and new operations, including their costs. The technical article proves one finite observation-simulation criterion, and states its limits explicitly.

3 Where the algebraic gain occurs

In one route through the OpenAI construction, a formula is converted into a polynomial H whose value at zero is the exact count. The algorithm computes Taylor data at other points and reconstructs the required value. These data do not arrive for free. They are obtained through measurements, substitutions, and sums over variable groups [5].

The reconstruction has a familiar information threshold. For arbitrary rational polynomials of degree at most d, initial Taylor coefficients at distinct nonzero points determine the value at zero exactly when their total number exceeds d. Below that threshold, adding the product of the required vanishing factors preserves all the observed coefficients while changing the value at zero.

That statement still does not determine the algorithm's running time. Nor does its necessity automatically hold within the narrower family of polynomials generated by formulas: the added polynomial may not belong to that family. A useful lower-bound argument must reach the actual computational construction.

The Claude method provides another route. It rewrites the counting problem as a network sum and uses exact identities to replace difficult crossings by sums of simpler pieces. Derived table values are evaluated by procedures, with their costs tracked recursively [1]. Its gain comes from how contributions are combined. An argument assuming that those procedures only inspect individual satisfying assignments would miss the mechanism.

The common question is what interaction remains expensive after the joint operation has been exploited. Recent analyses already address this question within specified methods: Dias and Lima study interpolation evaluation charges, Majdoddin studies restricted network recursions, and Tulsyan studies support-constrained counting templates [2, 3, 6]. Their permitted operations and cost models differ. The companion article compares them and supplies a fully checked finite analysis and measurement application for the swap and polynomial-fixing family. Its broader question remains how to make such structural explanations precise across representations.

4 A proved obstruction at the refined scale

The construction has a finite space indexed by histories of coordinate swaps. A polynomial in the swap-count operator is used to fix a large subspace. Measurements control the remaining directions. A smaller polynomial degree would reduce the substitutions needed later, but the measurement budget limits how many unfixed directions can be handled.

The theorem proves this tradeoff using the actual operators. Their spectrum has an explicit binomial distribution. A small number of measurement rows forces the fixing polynomial to agree with alternating signs on most of that distribution. Root counting and a binomial atom bound then force the polynomial's degree to be at least a constant times the square root of the number of swap positions.

The degree floor feeds into the specified recovery procedure. Write n for its variable budget, R for the fixing polynomial's positive degree budget, and $Q \geq 1$ for the number of swap positions. The recovery uses $q = \lfloor 3nR/Q \rfloor$. Its complete substitution family has size

$$A(n, q) = \sum_{j=0}^{\min(q,n)} 2^j \binom{n}{j}$$

Under the fixing condition and the construction's row and dimension budgets, the theorem gives

$$A(n, q) \geq 2^{n \log_2 \log_2 n/(4\sqrt{\log_2 n})} \qquad (n \geq 2^{16}).$$

It follows that for every fixed $\varepsilon > 0$, beyond a threshold depending only on ε,

$$2^{n^{1-\varepsilon}} \leq A(n, q) \leq 3^n.$$

The threshold is uniform over every permitted fixing polynomial, measurement map, and parameter choice. Thus changing the polynomial or measurement rows within these contracts cannot give the complete enumeration a fixed-power improvement in its exponent.

The source's sparse parity route uses different budgets and a different Taylor cutoff. The same degree argument still forces a near-exponential substitution family there, as the article shows [7, Section 5.5]. In both routes the result charges the routine that visits each term. A faster way to compute the aggregate, or a different family of operators, could change the cost.

The proof identifies a specific unresolved interaction and shows where its cost reappears: measurement capacity forces agreement, agreement forces degree, and degree forces enumeration. The spectral mechanism and the near-exponential template scale have public antecedents. The Lean development checks the finite chain and uniform eventual bound, constructs the physical rows selected by the source algorithm, and proves their fixing property, row count and height bounds [7]. The full formula-placement and counting algorithms retain their separate source analyses.

5 The broader claim remains a precise research question

The unrestricted #SH hypothesis concerns every deterministic exact-counting algorithm. The theorem above concerns one defined family. A proof of the former cannot be obtained just by removing the qualifications from the latter.

What the result supplies is an example of the kind of account a broader argument needs. It attaches hardness to task-changing alternatives and the actual operations used to resolve them. It allows a representation to produce a substantial gain, then proves a remaining tradeoff rather than assuming that the original cost survives unchanged.

The reported ETH algorithms make this distinction necessary. They show that an exponential-scale prediction can be too strong. They do not show that exact counting admits a fixed-power reduction in its exponent. Public successor hypotheses make that remaining claim legible and testable; construction-specific theorems can explain particular reasons it persists.

That is the direction developed in the companion paper: preserve the serious hardness question, refine its quantitative statement, and prove each structural obstruction at the scope its assumptions justify.

References

[1] Claude. A $2^{o(n)}$ time algorithm for 3-SAT, 2026. URL https://hexagonmath.org/2610.00577v1. Anthropic. Preprint dated 26 October 2026; hexagon:2610.00577v1, deposited 27 October 2026.

[2] Fabio Dias and Yvens Lima. Limits and constants of OpenAI's claimed ETH refutation, 2026. URL https://arxiv.org/abs/2611.00517v1. Public preprint, arXiv:2611.00517v1, 31 October 2026.

[3] Ruhollah Majdoddin. Lowered crossover points of Anthropic's subexponential 3-SAT algorithm, 2026. URL https://arxiv.org/abs/2610.41622v1. Public preprint, arXiv:2610.41622v1, 30 October 2026.

[4] MIT Hardness Group, Josh Brunner, Erik D. Demaine, Fabian Frei, Timothy Gomez, Markus Hecher, Jeffery Li, and Jayson Lynch. ETH is dead, long live SH, 2026. URL https://arxiv.org/abs/2611.02650v1. Public preprint, arXiv:2611.02650v1, 2 November 2026.

[5] OpenAI. The Exponential Time Hypothesis is false, 2026. URL https://github.com/openai/math/blob/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/build/main_full_version.tex. Preprint dated 17 October, revised 25 October 2026. Source snapshot 42907c9e999cc00d6abe7a50bfc1023250edd1b9.

[6] Vaibhav Tulsyan. What the ETH refutations do not give: barriers for the counting engines, with machine-checked lemmas, 2026. URL https://hexagonmath.org/2611.00149v1. Public preprint, hexagon:2611.00149v1, 6 November 2026.

[7] Rohan Vinaik. Refining hardness after ETH: Task-relevant distinctions and the cost of exact counting. Companion article and Lean development in this source package, 7 November 2026, 2026.